

AGENT INSTRUCTION: E-ITS IMR Step 1 - Scope Modeling (*Kaitseala modelleerimine*)

1. Objective & Architecture Overview

You are tasked with implementing the first and most critical stage of the Estonian Information Security Standard (E-ITS) workflow: **Scope Modeling (*Kaitseala modelleerimine*)**.

In this phase, the system maps security modules (*turvamoodulid*) from the E-ITS catalog (`eits_catalog_modules`) to specific assets (*sihtobjektid / IT-varad*) and business processes (*äriprotsessid*). Once mapped, the backend must automatically generate the corresponding implementation measures in the Infosec Measures Implementation Plan (IMR - *Infoturbe meetmete rakenduskava*) under `imr_items`.

System Data Flow:

```
[Business Processes] <--> [Assets (IT-varad)]
                        |
                [Mapped Modules (Kaardistatud moodulid)]
                        | (Auto-generates)
                [IMR Items (imr_items)]
```

Critical Business Rule: Protection Need Lockout (*Kaitsetarbe lukustamine*)

- **The Rule:** Once a Tenant's mode of protection (*turbeviis*: `BASELINE`, `STANDARD`, or `CORE`) has been selected and saved, the protection needs (*kaitsetarve*: Confidentiality, Integrity, Availability) of any related Business Processes **CANNOT** be modified.
- **Why:** This prevents security scope gaps where changing a protection need from "High" to "Normal" after modeling would silently mismatch the generated IMR items and violate compliance reporting.

2. Database Schema Design (SQL Migrations)

To bridge the gap between your assets (`assets`), business processes (`business_processes`), and the E-ITS catalog (`eits_catalog_modules`), we need to track which modules are assigned to which scope target.

We will implement a polymorphic mapping table `mapped_modules` to support assigning modules either directly to an **Asset** or to a **Business Process** (for organizational/process-oriented modules).

```
-- Migration: Add mapped_modules and support auto-generation relations
CREATE TABLE public.mapped_modules (
    id uuid DEFAULT gen_random_uuid() NOT NULL,
    tenant_id uuid NOT NULL,
    eits_module_id uuid NOT NULL,
    target_type character varying(50) NOT NULL, -- 'asset' või 'business_process'
    target_id uuid NOT NULL,                    -- Viide vastavale asset_id või b
    created_at timestamp with time zone DEFAULT now(),
```

```
CONSTRAINT pk_mapped_modules PRIMARY KEY (id)
```

```

CONSTRAINT pk_mapped_modules PRIMARY KEY (id),
CONSTRAINT fk_mapped_mod_tenant FOREIGN KEY (tenant_id) REFERENCES public.app
CONSTRAINT fk_mapped_mod_catalog FOREIGN KEY (eits_module_id) REFERENCES publ
CONSTRAINT uq_tenant_module_target UNIQUE (tenant_id, eits_module_id, target_
);

-- Index for polymorphic lookups
CREATE INDEX ix_mapped_modules_lookup ON public.mapped_modules USING btree (targe

-- Track which IMR item was generated by which mapped module for cleanup and sync
ALTER TABLE public.imr_items
ADD COLUMN IF NOT EXISTS mapped_module_id uuid,
ADD CONSTRAINT fk_imr_mapped_module FOREIGN KEY (mapped_module_id) REFERENCES pub

```

3. Backend Logic & Services (FastAPI / SQLAlchemy)

Implement the core service logic to handle module mapping, automatic IMR item generation, and protection need locks.

E-ITS Rule of Mode of Protection (*Turbeviis*):

- If the tenant's chosen mode of protection is **BASELINE** (*põhiturve*), generate IMR items only for measures (*eits_catalog_measures*) with level **BASE** (*põhimeetmed*).
- If the mode is **STANDARD** (*standardturve*) or **CORE** (*tuumikuturve*), generate IMR items for both **BASE** and **STANDARD** level measures.

A. Core Service Layer (`backend/app/services/v2/modeling_service.py`)

```

import uuid
from typing import List, Dict, Any
from sqlalchemy.orm import Session
from sqlalchemy import and_
from fastapi import HTTPException, status
# Impordi vajalikud andmebaasi mudelid siia (nt Asset, MappedModule, ImrItem, Bus

class ModelingService:
    @staticmethod
    def map_module_to_target(
        db: Session,
        tenant_id: uuid.UUID,
        user_id: uuid.UUID,
        module_id: uuid.UUID,
        target_type: str,
        target_id: uuid.UUID
    ) -> Dict[str, Any]:
        """
        Seob E-ITS mooduli sihtobjektiga (vara või äriprotsess) ja genereerib aut
        """
        # 1. Kontrolli, kas sihtobjekt kuulub aktiivsele tenantile
        if target_type == "asset":
            asset = db.query(Asset).filter(Asset.id == target_id, Asset.tenant_id
            if not asset:
                raise HTTPException(status_code=404, detail="Seda vara ei leitud
            elif target_type == "business_process":
                bp = db.query(BusinessProcess).filter(BusinessProcess.id == target_id
                if not bp:

```

```

else:
    raise HTTPException(status_code=400, detail="Vigane sihtobjekti tüüp.

# 2. Kontrolli, kas seos on juba olemas
existing_mapping = db.query(MappedModule).filter(
    MappedModule.tenant_id == tenant_id,
    MappedModule.eits_module_id == module_id,
    MappedModule.target_type == target_type,
    MappedModule.target_id == target_id
).first()

if existing_mapping:
    raise HTTPException(status_code=400, detail="See moodul on sellele ob

# 3. Loo uus seos (Kaitseala modelleerimine)
mapping = MappedModule(
    tenant_id=tenant_id,
    eits_module_id=module_id,
    target_type=target_type,
    target_id=target_id,
    created_by=user_id
)
db.add(mapping)
db.flush() # Genereerib ID ilma transaktsiooni lõpetamata

# 4. Tõmba tenandi praegune turbeviis (põhi-, standard- või tuumikuturve)
tenant_profile = db.query(TenantSecurityProfile).filter(TenantSecurityPro
mode_of_protection = tenant_profile.mode_of_protection if tenant_profile

# 5. Too kõik selle mooduliga seotud meetmed kataloogist
measures = db.query(EitsCatalogMeasure).filter(EitsCatalogMeasure.module_

generated_count = 0
for measure in measures:
    # Filtreeri meetmed vastavalt valitud turbeviisile
    # BASELINE korral võetakse ainult põhimeetmed (M ehk BASE)
    # STANDARD ja CORE korral võetakse nii põhimeetmed kui ka standardmee
    should_apply = False
    if mode_of_protection == "BASELINE" and measure.level == "BASE":
        should_apply = True
    elif mode_of_protection in ["STANDARD", "CORE"] and measure.level in
        should_apply = True

    if should_apply:
        # Kontrolli, kas meede on juba mingil põhjusel IMR-is olemas
        existing_imr = db.query(ImrItem).filter(
            ImrItem.tenant_id == tenant_id,
            ImrItem.eits_measure_id == measure.id,
            ImrItem.asset_id == (target_id if target_type == "asset" else
            ImrItem.business_process_id == (target_id if target_type == "
        ).first()

        if not existing_imr:
            # Loo uus rakenduskava meede (IMR meede) staatusega P (Planne
            imr_item = ImrItem(
                tenant_id=tenant_id,
                eits_measure_id=measure.id,
                mapped_module_id=mapping.id,
                asset_id=target_id if target_type == "asset" else None,
                business_process_id=target_id if target_type == "business
                status="P", # P - Kavandatud (Planned)

```

```

        description="", # Iun1, kasutaja talaab niijem
        created_by=user_id
    )
    db.add(imr_item)
    generated_count += 1

db.commit()
return {
    "message": "Moodul edukalt modelleeritud ja IMR-i kirjed genereeritud",
    "mapped_module_id": mapping.id,
    "generated_measures_count": generated_count
}

@staticmethod
def remove_module_from_target(db: Session, tenant_id: uuid.UUID, mapping_id:
    """
    Eemaldab mooduli kaitsealast ja kustutab automaatselt seotud IMR kirjed,
    """
    mapping = db.query(MappedModule).filter(MappedModule.id == mapping_id, Ma
    if not mapping:
        raise HTTPException(status_code=404, detail="Modelleeritud seost ei l

    # Eemaldame ainult need IMR kirjed, mis on alles 'Kavandatud' (P) staatus
    # Kui meede on juba rakendamisel (E) või tehtud (R), siis me ei kustuta s
    unmodified_imr_items = db.query(ImrItem).filter(
        ImrItem.mapped_module_id == mapping.id,
        ImrItem.status == "P"
    ).all()

    deleted_count = len(unmodified_imr_items)
    for item in unmodified_imr_items:
        db.delete(item)

    db.delete(mapping)
    db.commit()

    return {
        "message": "Moodul kaitsealast eemaldatud.",
        "deleted_imr_items_count": deleted_count
    }

@staticmethod
def update_process_protection_need(
    db: Session,
    tenant_id: uuid.UUID,
    process_id: uuid.UUID,
    new_confidentiality: str,
    new_integrity: str,
    new_availability: str
) -> Dict[str, Any]:
    """
    Uuendab äriprotsessi kaitsetarvet, KUID keelab selle kui turbeviis on jub
    """
    # Kontrolli, kas tenandil on turbeviis juba profiilis valitud/määratud
    tenant_profile = db.query(TenantSecurityProfile).filter(TenantSecurityPro

    if tenant_profile and tenant_profile.mode_of_protection is not None:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Kaitsetarvet ei saa muuta, sest turbeviis (põhi-, standar
        )

```

```

# teosta tavapärane kaitsetarve uuendamise loogika
process = db.query(BusinessProcess).filter(BusinessProcess.id == process_id)
if not process:
    raise HTTPException(status_code=404, detail="Äriprotsessi ei leitud.")

# Kaitsetarve uuendamine (Sõltub teie konkreetsest tabeli või seotud tabeli)
# Näide:
process.confidentiality_need = new_confidentiality
process.integrity_need = new_integrity
process.availability_need = new_availability

db.commit()
return {"message": "Äriprotsessi kaitsetarve edukalt uuendatud."}

```

B. API Controllers (backend/app/api/v2/endpoints/modeling.py)

```

from fastapi import APIRouter, Depends, Query, status, HTTPException
from sqlalchemy.orm import Session
import uuid
# Impordi autentimise dependencies ja andmebaasi sessioon

router = APIRouter(prefix="/modeling", tags=["E-ITS Kaitseala modelleerimine"])

@router.post("/map", status_code=201)
def map_module(
    module_id: uuid.UUID,
    target_type: str = Query(..., description="Kas 'asset' või 'business_process'"),
    target_id: uuid.UUID = Query(..., description="Vara või äriprotsessi unikaalne ID"),
    db: Session = Depends(get_db),
    current_user = Depends(get_current_active_user)
):
    """
    Seob E-ITS kataloogi mooduli sihtobjektiga ja loob automaatselt IMR meetmed.
    """
    return ModelingService.map_module_to_target(
        db=db,
        tenant_id=current_user.tenant_id,
        user_id=current_user.id,
        module_id=module_id,
        target_type=target_type,
        target_id=target_id
    )

@router.delete("/map/{mapping_id}", status_code=200)
def unmap_module(
    mapping_id: uuid.UUID,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_active_user)
):
    """
    Eemaldab mooduli sihtobjekti küljest ja kustutab algsed IMR kirjed.
    """
    return ModelingService.remove_module_from_target(
        db=db,
        tenant_id=current_user.tenant_id,
        mapping_id=mapping_id
    )

```

```
@router.patch("/{business-process/{process_id}/protection-need", status_code=200)
def update_process_needs(
    process_id: uuid.UUID,
    confidentiality: str,
    integrity: str,
    availability: str,
    db: Session = Depends(get_db),
    current_user = Depends(get_current_active_user)
):
    """
    Uuendab äriprotsessi C-I-A kaitsetarbeid. Tagastab vea, kui turbeviis on juba
    """
    return ModelingService.update_process_protection_need(
        db=db,
        tenant_id=current_user.tenant_id,
        process_id=process_id,
        new_confidentiality=confidentiality,
        new_integrity=integrity,
        new_availability=availability
    )
```

4. Frontend UI Component (React & Tailwind CSS)

Build a fully interactive modeling dashboard where users can map E-ITS modules to assets/processes and see changes instantly. Features protective lockout alerts when `modeOfProtection` is already defined.

```
import React, { useState, useEffect } from 'react';

// Tüübid andmevahetuseks
interface Asset {
    id: string;
    name: string;
    type: string;
}

interface EitsModule {
    id: string;
    code: string;
    name: string;
    description: string;
}

interface BusinessProcess {
    id: string;
    name: string;
    confidentiality: string;
    integrity: string;
    availability: string;
}

export default function App() {
    // Simuleeritud tenandi olek (nt. 'BASELINE', 'STANDARD' või null kui pole veel
    const [modeOfProtection, setModeOfProtection] = useState<string | null>('STANDA

    const [assets, setAssets] = useState<Asset[]>([
        { id: '1', name: 'Peamine andmebaas', type: 'Server' },
    ])
```

```

    });

const [processes, setProcesses] = useState<BusinessProcess[]>([
  { id: 'p1', name: 'Üldarstiabi osutamine', confidentiality: 'HIGH', integrity
  { id: 'p2', name: 'Töötajate värbamine', confidentiality: 'NORMAL', integrity
]);

const [modules, setModules] = useState<EitsModule[]>([
  { id: '101', code: 'SYS.1.1', name: 'Üldine server', description: 'Serverite
  { id: '102', code: 'SYS.3.1', name: 'Sülearvutid', description: 'Mobiilsete t
  { id: '103', code: 'CON.3', name: 'Andmevarundus', description: 'Regulaarse a
]);

const [selectedAsset, setSelectedAsset] = useState<string>('');
const [selectedModule, setSelectedModule] = useState<string>('');
const [mappedItems, setMappedItems] = useState<any[]>([]);
const [isLoading, setIsLoading] = useState<boolean>(false);
const [notification, setNotification] = useState<{ message: string; type: 'succ

// Protsessi muutmisega seotud vaated
const [activeProcess, setActiveProcess] = useState<BusinessProcess | null>(null

const handleMapModule = () => {
  if (!selectedAsset || !selectedModule) {
    showNotification('Palun vali nii sihtobjekt kui ka moodul!', 'error');
    return;
  }

  setIsLoading(true);

  setTimeout(() => {
    const asset = assets.find(a => a.id === selectedAsset);
    const mod = modules.find(m => m.id === selectedModule);

    const isAlreadyMapped = mappedItems.some(
      item => item.assetId === selectedAsset && item.moduleId === selectedModul
    );

    if (isAlreadyMapped) {
      showNotification('See moodul on sellele varale juba lisatud!', 'error');
      setIsLoading(false);
      return;
    }

    const newMapping = {
      id: Math.random().toString(36).substr(2, 9),
      assetId: selectedAsset,
      assetName: asset?.name,
      moduleId: selectedModule,
      moduleCode: mod?.code,
      moduleName: mod?.name,
      generatedMeasures: mod?.code === 'CON.3' ? 4 : 7
    };

    setMappedItems([...mappedItems, newMapping]);
    showNotification(
      'Edukalt seotud! Rakenduskavasse (IMR) lisati automaatselt ${newMapping.g
      'success'
    );
    setIsLoading(false);
  }, 800);

```

```

const handleUnmapModule = (id: string) => {
  setMappedItems(mappedItems.filter(item => item.id !== id));
  showNotification('Moodul eemaldatud ja alustamata IMR kirjed tühistatud.', 's
};

const handleTryUpdateProtectionNeed = (proc: BusinessProcess) => {
  if (modeOfProtection) {
    showNotification(
      `Viga: Äriprotsessi kaitsetarvet ei saa muuta, kuna turbeviis (${modeOfPr
      'error'
    );
    return;
  }
  setActiveProcess(proc);
};

const showNotification = (message: string, type: 'success' | 'error') => {
  setNotification({ message, type });
  setTimeout(() => setNotification(null), 5000);
};

return (
  <div className="min-h-screen bg-gray-50 py-8 px-4 sm:px-6 lg:px-8 font-sans">
    <div className="max-w-4xl mx-auto">

      {/* Päis */}
      <div className="bg-white rounded-lg shadow-sm p-6 mb-6 border border-gray
        <div className="flex justify-between items-start">
          <div>
            <h1 className="text-2xl font-bold text-gray-900">E-ITS Kaitseala mo
            <p className="mt-2 text-sm text-gray-600">
              Selles etapis seotakse E-ITS etalonturbe moodulid organisatsiooni
            </p>
          </div>
          <div className="flex flex-col items-end">
            <span className="text-xs font-bold text-gray-400 uppercase">Valitud
            <span className={`mt-1 px-3 py-1 rounded text-xs font-bold ${
              modeOfProtection ? 'bg-indigo-100 text-indigo-800 border border-i
            }`} >
              {modeOfProtection ? `LUKUSTATUD: ${modeOfProtection}` : 'Määramat
            </span>
          </div>
        </div>
      </div>

      {/* Teavitused */}
      {notification && (
        <div className={`p-4 rounded-md mb-6 transition-all border ${
          notification.type === 'success'
            ? 'bg-green-50 text-green-800 border-green-200'
            : 'bg-red-50 text-red-800 border-red-200'
        }`} >
          <p className="text-sm font-semibold">{notification.message}</p>
        </div>
      )}

      {/* Äriprotsesside kaitsetarve seisu paneel */}
      <div className="bg-white p-6 rounded-lg shadow-sm border border-gray-150
        <div className="flex justify-between items-center mb-4">
          <h2 className="text-lg font-semibold text-gray-800">Äriprotsesside ka

```

```

    <span className="text-xs text-red-500 font-medium flex items-center">
      🚫 Lukustatud turbeviisi tõttu
    </span>
  )}
</div>

<div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
  {processes.map(p => (
    <div key={p.id} className="p-4 rounded-lg bg-gray-50 border border-gray-100">
      <div>
        <h3 className="font-bold text-gray-900 text-sm">{p.name}</h3>
        <div className="mt-2 flex gap-2">
          <span className="text-xs font-bold bg-white border px-1.5 py-0.5">
            <span className="text-xs font-bold bg-white border px-1.5 py-0.5">
              <span className="text-xs font-bold bg-white border px-1.5 py-0.5">
                </div>
              </div>
            </div>
          <button
            onClick={() => handleTryUpdateProtectionNeed(p)}
            className={`mt-4 text-xs font-semibold py-1.5 px-3 rounded text-gray-700
              ${modeOfProtection === 'off' ? 'bg-gray-200 text-gray-400 cursor-not-allowed' : 'bg-indigo-50 text-indigo-700 hover:bg-indigo-100'}
            `>
              <div>
                {modeOfProtection === 'off' ? '🔒 Muutmine lukustatud' : 'Muuda kaitsetarvikut'}
              </div>
            </button>
          </div>
        )})}
      </div>
    </div>

    /* Modelleerimise paneel */
    <div className="grid grid-cols-1 md:grid-cols-2 gap-6 mb-6">
      <div className="bg-white p-6 rounded-lg shadow-sm border border-gray-15">
        <h2 className="text-lg font-semibold text-gray-800 mb-4">Loo uus seos

        <div className="mb-4">
          <label className="block text-xs font-bold text-gray-700 uppercase">Vali sihtobjekt (IT-vara)</label>
          <select
            className="w-full border border-gray-300 rounded-md p-2.5 bg-white"
            value={selectedAsset}
            onChange={(e) => setSelectedAsset(e.target.value)}>
            <option value="">-- Vali vara --</option>
            {assets.map(a => (
              <option key={a.id} value={a.id}>{a.name} ({a.type})</option>
            ))}
          </select>
        </div>

        <div className="mb-6">
          <label className="block text-xs font-bold text-gray-700 uppercase">Vali E-ITS turvamoodul</label>
          <select
            className="w-full border border-gray-300 rounded-md p-2.5 bg-white"
            value={selectedModule}
            onChange={(e) => setSelectedModule(e.target.value)}>
            <option value="">-- Vali moodul --</option>
            {modules.map(m => (
              <option key={m.id} value={m.id}>{m.name} ({m.type})</option>
            ))}
          </select>
        </div>
      </div>
    </div>
  )}
</div>

```

```

<option value="">-- vali moodul --</option>
{modules.map(m => (
  <option key={m.id} value={m.id}>{m.code} - {m.name}</option>
))}
</select>
</div>

<button
  onClick={handleMapModule}
  disabled={isLoading}
  className={`w-full text-white font-medium py-2.5 px-4 rounded-md te
    isLoading ? 'bg-blue-400 cursor-not-allowed' : 'bg-blue-600 hover
  }`}
>
  {isLoading ? 'Modelleerin ja genereerin IMR kirjeid...' : 'Kinnita
</button>
</div>

<div className="bg-white p-6 rounded-lg shadow-sm border border-gray-15
  <div>
    <h2 className="text-lg font-semibold text-gray-800 mb-2">Juhised ja
    <ul className="list-disc list-inside text-xs text-gray-600 space-y-
      <li>Süsteem tuvastab tenandi seadistustest automaatselt valitud <
      <li><strong>Põhiturbe (BASELINE)</strong> korral luuakse IMR-i ai
      <li><strong>Standardturbe</strong> korral lisatakse automaatselt
      <li>Seose eemaldamisel tühistatakse ainult alustamata (Planned) m
      <li className="text-red-600 font-semibold">Kui turbeviis on määra
    </ul>
  </div>
  <div className="mt-6 pt-4 border-t border-gray-100 bg-gray-50 p-3 rou
    Maksimumväärtuse printsiip: Kõrge kaitsetarbe korral nõutakse raken
  </div>
</div>
</div>

{ /* Kaardistatud kaitseala nimekiri */ }
<div className="bg-white rounded-lg shadow-sm border border-gray-150 over
  <div className="px-6 py-4 border-b border-gray-150 bg-gray-50 flex just
    <h2 className="text-md font-bold text-gray-800">Kaardistatud kaitseal
    <span className="text-xs font-semibold bg-blue-100 text-blue-800 px-2
      E-ITS Juurutusfaas
    </span>
  </div>
</div>

{mappedItems.length === 0 ? (
  <div className="p-8 text-center text-sm text-gray-500">
    Kaitseala on tühi. Alusta modelleerimist lisades mõni moodul sihtob
  </div>
) : (
  <div className="divide-y divide-gray-150">
    {mappedItems.map((item) => (
      <div key={item.id} className="p-6 flex items-start justify-betwee
        <div>
          <span className="inline-flex items-center px-2 py-0.5 rounded
            {item.moduleCode}
          </span>
          <h3 className="text-sm font-semibold text-gray-900">{item.mod
          <p className="text-xs text-gray-500 mt-1">
            Sihtobjekt: <strong className="text-gray-700">{item.assetNa
          </p>
          <p className="text-xs text-green-600 mt-2 font-medium">

```

```

        </p>
      </div>
      <button
        onClick={() => handleUnmapModule(item.id)}
        className="text-xs text-red-600 hover:text-red-800 font-semib
      >
        Eemalda kaitsealast
      </button>
    </div>
  )}}
</div>
)}
</div>

</div>
</div>
);
}

```

5. Backend Integration Tests (Pytest)

Ensure correct functionality, transaction safety, and tenant isolation using this test suite.

```

# test_modeling_service.py
import pytest
import uuid
from sqlalchemy.orm import Session
from fastapi import HTTPException
# Impordi db_session fixture, test-mudelid ja teenuseklassid

def test_map_module_enforces_tenant_isolation(db_session: Session):
    """
    Testib, et kasutaja ei saaks siduda moodulit teise tenandi varaga.
    """
    tenant_a_id = uuid.uuid4()
    tenant_b_id = uuid.uuid4()
    user_a_id = uuid.uuid4()

    # Loo teise tenandi vara
    unauthorized_asset = Asset(
        id=uuid.uuid4(),
        tenant_id=tenant_b_id,
        name="Võõras Server",
        type="Server"
    )
    db_session.add(unauthorized_asset)

    # Loo testitav moodul
    eits_module = EitsCatalogModule(
        id=uuid.uuid4(),
        code="SYS.1.1",
        name="Testmoodul"
    )
    db_session.add(eits_module)
    db_session.commit()

    # Proovi teostada sidumist teise tenandi objektiga

```

```

        modeling_service.map_module_to_target(
            db=db_session,
            tenant_id=tenant_a_id,
            user_id=user_a_id,
            module_id=eits_module.id,
            target_type="asset",
            target_id=unauthorized_asset.id
        )

    assert exc_info.value.status_code == 404
    assert "Seda vara ei leitud" in exc_info.value.detail

def test_map_module_generates_imr_items_based_on_protection_mode(db_session: Session):
    """
    Kontrollib, et IMR meetmete genereerimisel arvestatakse tenandi turbeviisi.
    """
    tenant_id = uuid.uuid4()
    user_id = uuid.uuid4()
    asset_id = uuid.uuid4()
    module_id = uuid.uuid4()

    # Loo testandmed
    tenant_profile = TenantSecurityProfile(
        tenant_id=tenant_id,
        mode_of_protection="BASELINE" # Ainult põhiturve
    )
    asset = Asset(id=asset_id, tenant_id=tenant_id, name="Test server", type="Server")
    module = EitsCatalogModule(id=module_id, code="SYS.1.1", name="Test moodul")

    # Loo üks põhimeede ja üks standardmeede
    base_measure = EitsCatalogMeasure(id=uuid.uuid4(), module_id=module_id, code="BASE")
    standard_measure = EitsCatalogMeasure(id=uuid.uuid4(), module_id=module_id, code="STANDARD")

    db_session.add_all([tenant_profile, asset, module, base_measure, standard_measure])
    db_session.commit()

    # Testi BASELINE loogikat
    result = ModelingService.map_module_to_target(
        db=db_session,
        tenant_id=tenant_id,
        user_id=user_id,
        module_id=module_id,
        target_type="asset",
        target_id=asset_id
    )

    # Ainult põhimeede peaks olema loodud
    assert result["generated_measures_count"] == 1

    created_imr = db_session.query(ImrItem).filter(ImrItem.tenant_id == tenant_id)
    assert len(created_imr) == 1
    assert created_imr[0].eits_measure_id == base_measure.id
    assert created_imr[0].status == "P" # Vaikimisi 'Planned' staatus

def test_update_protection_need_locked_when_mode_selected(db_session: Session):
    """
    Kontrollib, et kui turbeviis on valitud (mode_of_protection != None),
    visatakse äriprotsessi kaitsetarbe muutmisel viga (HTTP 400).
    """

```

```

process_id = uuid.uuid4()

# Loo profiil kus turbeviis on valitud
tenant_profile = TenantSecurityProfile(
    tenant_id=tenant_id,
    mode_of_protection="STANDARD"
)
process = BusinessProcess(
    id=process_id,
    tenant_id=tenant_id,
    name="Testprotsess",
    confidentiality_need="NORMAL",
    integrity_need="NORMAL",
    availability_need="NORMAL"
)
db_session.add_all([tenant_profile, process])
db_session.commit()

# Proovime uuendada kaitsetarvet, oodates viga
with pytest.raises(HTTPException) as exc_info:
    ModelingService.update_process_protection_need(
        db=db_session,
        tenant_id=tenant_id,
        process_id=process_id,
        new_confidentiality="HIGH",
        new_integrity="HIGH",
        new_availability="HIGH"
    )

assert exc_info.value.status_code == 400
assert "Kaitsetarvet ei saa muuta" in exc_info.value.detail

```

6. Execution Steps for the Coding Agent

1. **Alembic Migration:** Run the Alembic commands to update the database schema with the `mapped_modules` table.
2. **FastAPI Services:** Save the `ModelingService` to your services folder, implementing both modeling mappings and the new protection need lock logic. Register the routes inside the API v2 routing chain.